# Sparse Latent Dynamics for Financial Forecasting under Market Shifts


| Zhijie Jiang | Boyu Song | Shuo Liu | Yanming Zhao |
|---|---|---|---|
| jian1223 | songboy3 | liushu70 | zhaoy439 |



## Abstract

Financial returns are noisy and nonstationary, making it difficult for sequence models to generalize across market conditions. We propose to test whether sparse latent dynamics improve next-day ETF return forecasting relative to a standard gated recurrent unit (GRU). A GRU encodes a fixed historical window into a low-dimensional state; a learned state transition is regularized using a small polynomial library and a sparsity penalty. We will compare a direct GRU, a dense latent-transition model, and a sparse latent-transition model using chronological out-of-sample evaluation on four liquid ETFs. Performance will be assessed with forecast errors and regime-stratified diagnostics. We will also examine whether discovered terms remain stable when the encoder is held fixed. The primary contribution is a controlled evaluation of a dynamical inductive bias, not a claim to discover universal financial laws. Regime-conditioned transitions and more elaborate rolling retraining are optional extensions.


## 1 Introduction

Financial return predictability can change as volatility, monetary conditions, and cross-asset relationships evolve. Recurrent neural networks can learn nonlinear patterns from sequential observations, but may overfit historical relationships. Sparse identification of nonlinear dynamics (SINDy) offers a complementary structural assumption: a low-dimensional state transition may be approximated by a small number of candidate terms (Brunton et al., 2016).

Our question is: *Does a sparse latent-state transition improve out-of-sample financial forecasting relative to an otherwise comparable GRU, particularly during volatile market periods?* We do not assume that prices obey a deterministic ordinary differential equation. Instead, we test whether a parsimonious approximate transition is a useful predictive inductive bias.

**Input and output.** For trading day $t$, the input is a lookback window $X_t = [x_{t-L+1}, \ldots, x_t] \in \mathbb{R}^{L \times d_x}$ of historical features observed by the end of day $t$, with $L = 40$ initially. The output is a vector $\hat{y}_{t+1} \in \mathbb{R}^N$ predicting next-day close-to-close log returns for $N = 4$ ETFs. The prediction time is after the close on day $t$; any feature not available by then is excluded.

**Planned contributions.**

1. A reproducible chronological evaluation of GRU forecasting on a small multi-asset panel.
2. A controlled comparison of direct GRU, dense latent transition, and sparse latent transition, using the same sequence encoder where possible.
3. An analysis of forecast errors in calm versus stressed markets and of sparse-term stability in a fixed latent coordinate system.

Preprint.

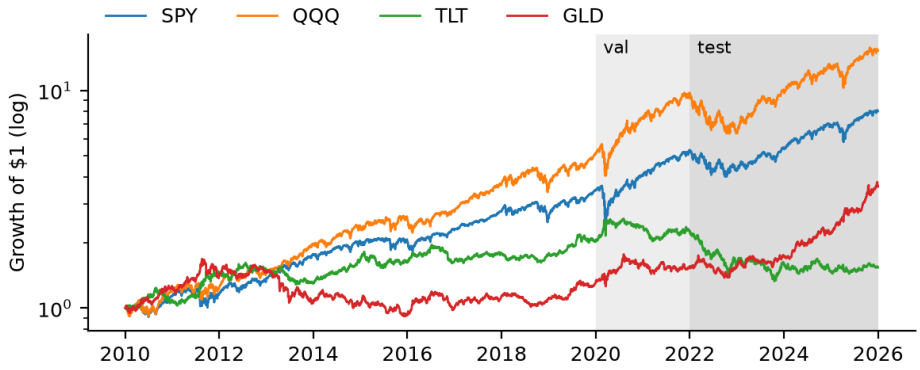



Figure 1: Value over time of $1 invested in each ETF at the start of 2010, plotted on a logarithmic vertical axis. The unshaded region (2010–2019) is the training period; light-gray and darker-gray backgrounds identify the validation and test periods. The curves show cumulative performance, whereas our prediction targets are daily log returns.

## 2 Background and Related Work

**Financial sequence models.** Fischer and Krauss (Fischer and Krauss, 2018) evaluate LSTM models for financial prediction, motivating recurrent encoders for historical return sequences. Our study uses a GRU and emphasizes controlled comparisons rather than a more complex predictive architecture.

**Sparse dynamics discovery.** SINDy (Brunton et al., 2016) represents state changes using a sparse combination of candidate nonlinear terms. Champion et al. (Champion et al., 2019) jointly learn latent coordinates and governing equations with an autoencoder. SINDy-SHRED (Gao et al., 2026) combines recurrent representations with sparse latent dynamics. Thus, combining recurrent learning and SINDy is *not* claimed as a new general method; we examine its usefulness for nonstationary financial prediction.

**Nonstationarity and alternative dynamics.** Wang and Lera (Wang and Lera, 2026) study return prediction under shifting market regimes. Nguyen et al. (Nguyen et al., 2024) investigate neural ODEs for stock forecasting. Unlike a neural-ODE approach, our core method uses a one-day discrete transition and does not require a numerical ODE solver. We focus on whether sparsity provides benefits beyond adding a transition layer.

## 3 Data

**Source and coverage.** We downloaded daily adjusted closing prices and trading volumes for SPY, QQQ, TLT, and GLD from Yahoo Finance using `yfinance` with automatic price adjustment enabled. These ETFs provide exposure to U.S. large-cap equities, Nasdaq-100 equities, long-term U.S. Treasury bonds, and gold, respectively. Prices span January 4, 2010 to December 31, 2025. Computing daily log returns yields 4,023 observations from January 5, 2010 onward. The downloaded closing-price panel has no missing values across the four ETFs. Figure 1 summarizes their historical performance; to make the curves easy to compare, it shows how much an initial investment of $1 in each ETF would be worth over time.

**Inputs and prediction targets.** For asset $i$, the target is $r_{t+1}^{(i)} = \log(P_{t+1}^{(i)}/P_t^{(i)})$, where $P_t^{(i)}$ is the adjusted closing price. Each input will contain 40 consecutive trading days of features available by the close of day $t$; the output will contain the four next-day log returns. Candidate features include historical returns, trailing volatility, momentum, and relative trading volume. We have not decided

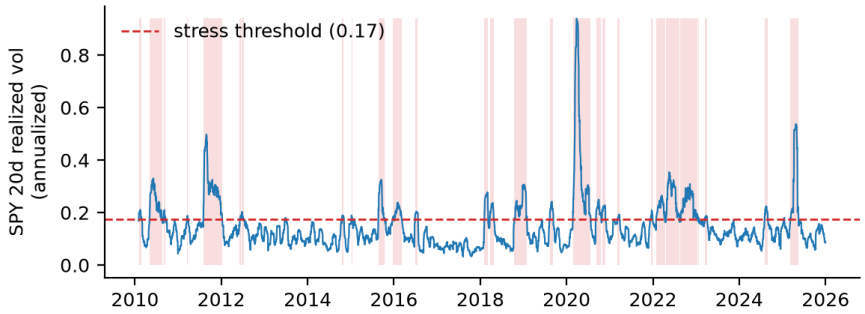



Figure 2: SPY trailing 20-day volatility (annualized). The dashed line marks the stress threshold of 17.4%, set at the 80th percentile of the training period; shaded dates exceed this threshold.

yet on the exact definition of each feature or how many past days it should look back over; we will choose these using the training and validation data, without looking at the test set.

**Preprocessing and chronological splits.** We use 2010–2019 as the training set, 2020–2021 as the validation set for hyperparameter tuning, and 2022–2025 as the held-out test set. These periods contain 2,515, 505, and 1,003 daily return observations, respectively, before constructing model inputs. If a sample does not have enough past data to build its features, we simply drop it instead of filling the gap with future values. Anything we compute for preprocessing, such as scaling parameters or clipping thresholds, comes from the training set only. To handle input windows that cross a split boundary, we assign each sample to the split of the day it predicts. This means a validation or test input can still look back at days from the previous split, since those days would already be known at prediction time. We will report the final number of usable samples once the features are built. The plots in this section use all years, because they only describe the data and are not used to train or tune any model. However, we will not look at test-set errors until we have finished choosing our model; otherwise, test results could influence our design choices.

**Volatility and evaluation regimes.** To tell calm markets apart from volatile ones, we measure how much SPY has been moving recently: for each day, we take the standard deviation of SPY's log returns over the past 20 trading days and multiply it by $\sqrt{252}$ to express it on a yearly scale. Here 252 is the approximate number of trading days in a U.S. market year. Taking the square root follows the standard square-root-of-time convention in finance: if daily returns are roughly independent, variance grows linearly with time, so the standard deviation grows with its square root. We then set a stress threshold of 17.4%, which is the 80th percentile of this value during the training period. Days above the threshold are labeled stressed, and the rest are labeled calm. The first 19 days of the dataset are left unlabeled, since they do not have 20 days of history yet. As Figure 2 shows, volatility changes a lot over time, so a single average error could hide how a model behaves in rough markets. We therefore report errors separately for calm and stressed days. These labels are only used to group errors during evaluation; we do not train separate models on them. Since a forecast for tomorrow is made today, we label each forecast using the market condition on the day the forecast is made.

**Temporal dependence and modeling motivation.** Figure 3 looks at whether today's market movement tells us anything about future movement. The autocorrelation of returns asks: if SPY went up today, is it more likely to go up (or down) after a given number of days? The autocorrelation of absolute returns ignores the direction and asks a different question: if SPY moved a lot today, in either direction, is it likely to move a lot again later?

For example, at a lag of one day, the return autocorrelation is about $-0.10$. This is close to zero, so knowing whether SPY went up or down today tells us very little about whether it will go up or down tomorrow. In contrast, the autocorrelation of absolute returns is about $0.31$, and it stays positive over the 30 lags shown. This means a big move today makes a big move tomorrow noticeably more likely,

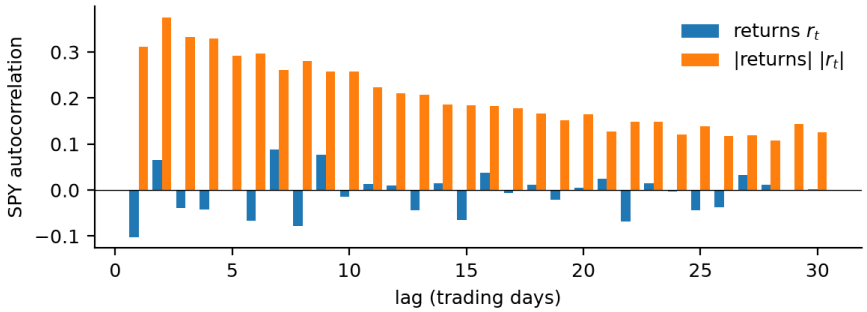



Figure 3: Autocorrelations of SPY daily log returns (blue) and absolute log returns (orange) at lags of 1 to 30 trading days. Each bar is computed from all pairs of days that are that many trading days apart, using the full 2010–2025 sample.

even if we cannot tell which way it will go. This pattern is known as volatility clustering: calm days tend to follow calm days, and turbulent days tend to follow turbulent days.

This pattern motivates our use of a sequence encoder, since the model may be able to learn this kind of lasting market state from the past 40 days. Our experiments will then answer three questions: whether the direction of returns can be predicted at all, whether the latent transition is actually sparse, and whether a sparse transition forecasts better than the direct-GRU and dense-transition baselines.

## 4 Model Architecture

**Shared GRU encoder.** A one-layer GRU with 64 hidden units maps each input window to a latent state:

$$z_t = E_\theta(X_t) \in \mathbb{R}^m, \qquad m = 4. \tag{1}$$

The same input features and encoder configuration will be used across learned baselines.

**Three principal models for comparison.** We will test 3 approaches on benchmarks: *direct GRU* as blank control, *dense-transition GRU* and *SINDy*. The *direct GRU* predicts $\hat{y}_{t+1} = h_\psi(z_t)$. The *dense-transition GRU* uses $\hat{z}_{t+1} = z_t + F_\phi(z_t)$ with a small MLP $F_\phi$, followed by $\hat{y}_{t+1} = h_\psi(\hat{z}_{t+1})$. The *sparse-transition GRU* instead uses

$$\hat{z}_{t+1} = z_t + \Theta(z_t)\Xi, \qquad \hat{y}_{t+1} = h_\psi(\hat{z}_{t+1}), \tag{2}$$

where $\Theta(z_t)$ contains constant, linear, and quadratic monomial candidates and $\Xi$ is a trainable coefficient matrix. All models predict next-day returns; only the transition model differs.

**Objective and identifiability.** The sparse model minimizes

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda_d \|E_\theta(X_{t+1}) - \hat{z}_{t+1}\|_2^2 + \lambda_s \|\Xi\|_1 + \lambda_r \mathcal{L}_{\text{rec}}. \tag{3}$$

Here $\mathcal{L}_{\text{pred}}$ is return MSE for comparing difference of predicted and actual returns, and $\mathcal{L}_{\text{rec}}$ is an optional lightweight reconstruction loss to discourage meaningless latent space learning. The dense-transition baseline uses the corresponding transition-consistency term but no sparse coefficient penalty. Since jointly learned latent coordinates are not unique, we will not directly compare coefficients from independently retrained encoders. For interpretability tests, we will freeze a common encoder and fit or evaluate transitions in that fixed coordinate system.

**Implementation.** We will use PyTorch, AdamW, early stopping on validation MSE, and a small validation-selected grid of $\lambda_d$ and $\lambda_s$. Initial settings are a 40-day window, 64 GRU hidden units, latent dimension four, and batch size 64; these may be adjusted using validation data. Neural ODEs and regime-specific coefficient mixtures are not required for the core project and would be attempted only with sufficient time.

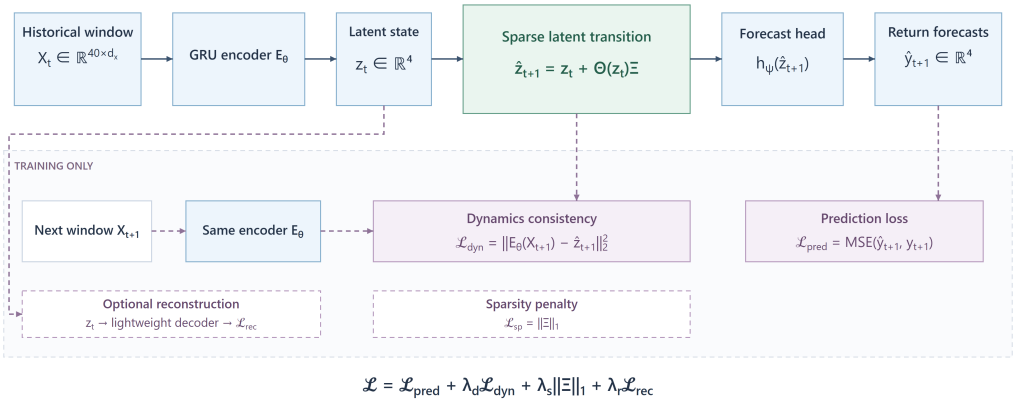



Figure 4: Architecture of the sparse latent-transition GRU. A one-layer GRU with 64 hidden units and a linear projection encodes a 40-day historical feature window into a four-dimensional latent state. A residual sparse transition predicts the next latent state using 15 constant, linear, and quadratic candidate terms, with coefficient matrix $\Xi \in \mathbb{R}^{15\times 4}$. The forecast head maps this predicted state to next-day log returns for SPY, QQQ, TLT, and GLD. Solid arrows denote the forward prediction path; dashed arrows denote training-only supervision. The next window is encoded using shared parameters to construct the dynamics-consistency target. Training combines return MSE, dynamics consistency, an $\ell_1$ coefficient penalty, and optional lightweight reconstruction. Future observations are used only as training targets; inference requires only $X_t$. The direct-GRU baseline predicts from $z_t$, while the dense-transition baseline replaces $\Theta(z_t)\Xi$ with a small MLP $F_\phi(z_t)$.

## 5 Proposed Evaluation

**Baselines.** We will report a zero-return predictor as a simple financial benchmark and compare three learned models: direct GRU, GRU with a dense latent transition, and GRU with a sparse latent transition. A generic weight-decay control will be included if resources permit.

**Primary metrics.** The main metrics are next-day return MSE, MAE, and directional accuracy. We will also report errors separately for predeclared calmer and stressed periods. All models will use identical chronological splits and feature availability assumptions.

**Dynamics diagnostics.** We will report the number of retained polynomial terms, one-step latent transition error, and whether active terms are stable across time blocks when using a frozen encoder. Because sparsity alone does not establish economic interpretability, we will describe discovered terms as model-dependent relationships rather than governing laws.

**Reproducibility and optional extensions.** We plan to run three seeds for the main models and report means and variability. If the core experiments finish early, we may add one rolling-origin refit, a transaction-cost-aware illustrative backtest, or regime-conditioned transition coefficients. These are explicitly secondary and will not be necessary to answer the primary question.

## 6 Ethical Considerations

Historical market prediction can be mistaken for investment advice. We will present results as a methodological comparison, not a deployable trading strategy. Retrospective tests may overstate usefulness due to data selection, transaction costs, and changing market conditions. We will report negative results and uncertainty, and avoid interpreting latent equations as causal or universal financial laws. Public-data licensing and reproducibility will be documented.

## 7 Work Division

The project will be completed by four students. The following provisional assignments will be confirmed by the team:

- **Shuo Liu:** download and validate ETF data, implement chronological splits, prevent leakage, and prepare exploratory figures.
- **Boyu Song:** implement the direct GRU and zero-return baseline, tune the GRU on validation data, and maintain reproducible training scripts.
- **Zhijie Jiang:** implement dense and sparse latent transitions, consistency losses, and sparsity diagnostics.
- **Yanming Zhao:** coordinate model comparisons, regime-stratified evaluation, plots, and assembly of the proposal and final report.

All members will review experimental design, test integrated code, interpret results, and contribute to writing. We plan a weekly team meeting and a shared repository with documented responsibilities.

## References


S. L. Brunton, J. L. Proctor, and J. N. Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937, 2016. DOI: 10.1073/pnas.1517384113.

K. Champion, B. Lusch, J. N. Kutz, and S. L. Brunton. Data-driven discovery of coordinates and governing equations. *Proceedings of the National Academy of Sciences*, 116(45):22445–22451, 2019. DOI: 10.1073/pnas.1906995116.

M. L. Gao, J. P. Williams, and J. N. Kutz. Sparse identification of nonlinear dynamics and Koopman operators with Shallow Recurrent Decoder Networks. *Proceedings of the National Academy of Sciences*, 123(16):e2508144123, 2026. DOI: 10.1073/pnas.2508144123.

T. Fischer and C. Krauss. Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2):654–669, 2018. DOI: 10.1016/j.ejor.2017.11.054.

Y. Wang and S. C. Lera. Meta-learning for return prediction in shifting market regimes. *Journal of Financial Markets*, 79:101042, 2026. DOI: 10.1016/j.finmar.2025.101042.

A. Q. Nguyen, S. Ha, and H. Thai. Phase space reconstructed neural ordinary differential equations model for stock price forecasting. In *Proceedings of the Pacific Asia Conference on Information Systems (PACIS)*, 2024.